NOM : Matricule :

PRÉNOM : SECTION :

Compléments d'Algorithmique – Examen du 27 août 2025

Consignes

- Commencez par indiquer votre nom, prénom, section et matricule au-dessus de chaque feuille.
- Répondez aux questions **directement sur les feuilles d'énoncés**.
- Les notes de cours et personnelles **ne sont pas autorisées**.
- Si ce n'est pas explicitement demandé dans l'énoncé :
  - **vous ne devez pas tester que les préconditions sont respectées** ;
  - **vos fonctions ne doivent pas avoir d'effet de bord** : les objets mutables passés en paramètres ne doivent pas être modifiés et rien ne doit être affiché à l'écran.

Problème 1 : Petites questions diverses

Soyez précis·e mais concis·e pour répondre aux questions suivantes.

Question 1.a. Dans une GUI conçue avec TKinter, qu'est-ce qu'un widget ? Donnez un exemple.

Réponse

Question 1.b. Expliquez brièvement le principe du **Test de Turing**

Réponse

Question 1.c. Donnez deux dangers possibles des IA génératives relatifs aux **libertés fondamentales**.

Réponse

Question 1.d. Expliquez comment il est possible d'avoir un accès à un élément d'une liste en Python en $O(1)$, alors que les données peuvent être hétérogènes.

Réponse

Question 1.e. Comment décririez-vous les principes et particularités de la **programmation fonctionnelle** ? Donnez un exemple typique de code Python inspiré de la programmation fonctionnelle.

Réponse

Problème 2 : Optimisation culinaire

Swen et Toby sont tellement fans de jeux qu'ils *gamifient* toutes leurs activités ensemble. Ils se sont préparés à manger et à boire mais décident de jouer à un jeu pour savoir qui a le droit de prendre quoi.

La nourriture et les boissons sont disposées en une rangée. Chacun son tour, en commençant par Swen, le joueur courant choisit soit ce qui se trouve le plus à gauche, soit ce qui se trouve le plus à droite, avant de laisser sa place à l'autre joueur. Chacun souhaite bien évidemment maximiser ses gains en prenant ce qui l'intéresse le plus.

Swen décide de profiter de ses compétences en programmation pour écrire une fonction qui lui renverra le meilleur gain possible pour toute rangée donnée. Il ne veut pas trop tricher, donc il se contente de cette information sans chercher à obtenir la solution qui mène à ce gain.

Pour ce faire, il représente la rangée sous la forme d'une liste de valeurs qu'il a attribuées aux aliments. Par exemple, la liste $[3, 20, 7, 5]$ pourrait représenter une rangée de 4 aliments avec respectivement une valeur attribuée de 3, 20, 7, 5. Au premier tour, on ne pourrait donc que sélectionner l'aliment qui vaut 3 ou celui qui vaut 5. Le meilleur gain possible pour cette rangée est de 25 (on prend 5, puis, peu importe ce qui est pris par l'autre joueur, on prend 20). Pour se simplifier la tâche, il part du principe que Toby a les mêmes envies que lui (c'est à dire qu'il utilise la même liste de valeurs).

Il programme alors une fonction récursive, qui prend 3 paramètres en entrée. Le premier paramètre $food$ est une liste telle que décrite ci-dessus, le deuxième paramètre l est un entier désignant l'indice de l'élément à prendre à gauche et le troisième paramètre r est un entier désignant l'indice de l'élément à prendre à droite.

Le cas de base de cette fonction récursive arrive lorsque $l = r$, ce qui veut dire qu'il ne reste plus qu'un élément à sélectionner. On sélectionne alors dans ce cas l'élément en retournant sa valeur. Il est également possible que l'indice l dépasse l'indice r, dans quel cas on renvoie 0.

Sinon, il y a deux possibilités, soit on sélectionne l'élément le plus à gauche, soit on sélectionne l'élément le plus à droite. Dans tous les cas, l'adversaire sélectionnera ensuite l'élément qui nous arrangera le moins. En effet, maximiser son propre gain revient à minimiser celui de l'adversaire (et donc, inversement, pour l'adversaire, maximiser son gain est équivalent à minimiser le notre).

En considérant cela, si on prend l'élément le plus à gauche, alors notre gain total correspond au gain de l'élément de gauche plus le minimum de gain restant après les deux choix possibles pour l'adversaire (c'est à dire, soit enlever un deuxième élément à gauche, soit enlever un élément à droite). De la même manière, si on prend l'élément le plus à droite, alors notre gain total correspond au gain de l'élément le plus à droite plus le minimum de gain restant après les deux choix possibles pour l'adversaire (c'est à dire, soit enlever un deuxième élément à droite, soit enlever un élément à gauche). Veillez donc à utiliser les bons arguments pour ces 4 appels récursifs, en faisant augmenter (ou non) l et en faisant diminuer (ou non) r.

Finalement, nous avons deux gains totaux différents selon que nous prenions l'élément de gauche ou de droite. Puisque nous souhaitons maximiser notre gain, nous retournons le maximum parmi les deux.

Question 2.a. Implémentez la fonction **récursive** `optimize(food,l,r)` qui prend en paramètres $food$ la liste contenant les valeurs associées aux éléments à choisir, l l'indice de l'élément à sélectionner à gauche, r l'indice de l'élément à sélectionner à droite. Cette fonction renvoie le gain total maximal possible pour cette liste de valeurs. Cette fonction doit **obligatoirement** suivre le fonctionnement donné ci-dessus.

Réponse *(Utilisez les lignes pour spécifier les indentations)*

Swen observe son travail et se rend compte que certaines informations sont calculées plusieurs fois, mais ne sait pas comment y remédier. Vous lui proposez de lui expliquer comment vous pouvez modifier sa fonction pour y intégrer de la mémoïsation.

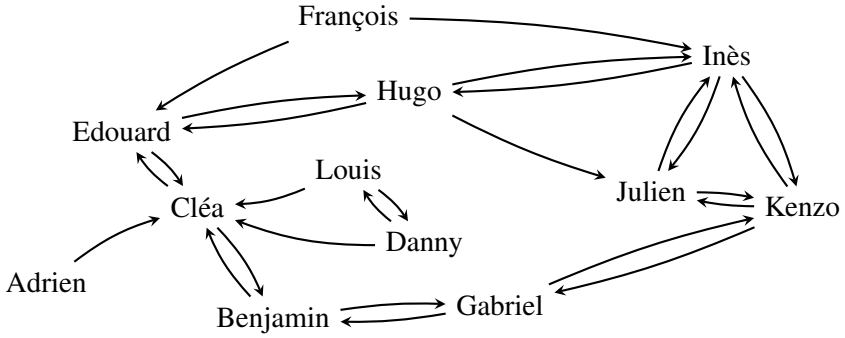
Question 2.b. Vous décidez d'utiliser un dictionnaire pour mettre en place la mémoïsation pour éviter de faire plus d'une fois le même calcul. Commencez par indiquer quelles vont être les paires (clé,valeur) que vous allez utiliser. Ensuite, indiquez les changements que vous effectuez sur la fonction de base. N'hésitez pas à indiquer des numéros de ligne dans votre réponse à la Q2.a si cela vous aide.

Réponse *(Utilisez les lignes pour spécifier les indentations)*

Problème 3 : Réseaux sociaux

Dans certains réseaux sociaux, les utilisateurs peuvent suivre d'autres utilisateurs, à la manière de plateformes comme Twitter ou Instagram. Cela signifie que chaque utilisateur peut choisir de s'abonner à d'autres, sans que ce soit forcément réciproque.

La figure ci-dessous illustre un exemple de réseau social :
— Chaque nom représente un utilisateur.
— Une flèche de u_1 vers u_2 signifie que l'utilisateur u_1 suit l'utilisateur u_2.



Certains utilisateurs peuvent se suivre mutuellement, comme c'est le cas de Benjamin et Gabriel, entre lesquels il y a deux flèches (une dans chage sens). Ce réseau peut être représenté mathématiquement par deux ensembles :
— U : l'ensemble des utilisateurs,
— $S \subseteq U \times U$: l'ensemble des relations de suivi. Un couple $(u_1, u_2) \in S$ signifie que l'utilisateur u_1 suit l'utilisateur u_2.

Par exemple, pour le réseau ci-dessus, on a :

$U = \{\text{Adrien, Benjamin, Cléa, } \dots\}$ et $S = \{(\text{Adrien, Cléa}), (\text{Cléa, Benjamin}), (\text{Benjamin, Cléa}), \dots\}$

Question 3.a. Quel utilisateur est le plus suivi dans le réseau ci-dessus ? Autrement dit, quel est celui qui a le plus grand nombre d'utilisateurs le suivant ?

Réponse

L'objectif de cette question est de modéliser de tels réseaux avec des structures de données et de concevoir un algorithme permettant de déterminer l'utilisateur le plus suivi.

Question 3.b. Quelles sont, selon vous, les deux structures de données les plus adaptées pour :
— représenter les utilisateurs ;
— représenter les relations de suivi ?

Votre choix ne doit pas forcément correspondre à la modélisation mathématique en termes d'ensembles. Choisissez ce qui vous semble le plus pratique pour implémenter l'algorithme de la question suivante.

Réponse

Structures de données pour les utilisateurs :

Structure de données pour les relations de suivi :

Question 3.c. Implémentez la fonction `plus_suivi(U, S)` retournant l'utilisateur le plus suivi dans le réseau social modélisé par les structures de données U et S que vous avez choisies à la Q3.b. Si plusieurs utilisateurs sont les plus suivis, la fonction retourne l'un d'eux.

Réponse *(Utilisez les lignes pour spécifier les indentations)*

Problème 4 : Des XOR à grande échelle

Lors de vos habituelles recherches sur Internet, vous apprenez que l'opérateur XOR est l'un des opérateurs de base de beaucoup d'opérateur de cryptographie. Pour rappel, XOR est le nom commun d'un opérateur "OU exclusif", souvent noté $\oplus$ ($0 \oplus 0 = 0$; $1 \oplus 0 = 1$; $0 \oplus 1 = 1$; $1 \oplus 1 = 0$; où 1 représente *vrai* et 0 représente *faux*). Vous apprenez qu'utiliser XOR seul comme approche cryptographique est une très mauvaise pratique car facilement cassable. Cependant, il a une propriété cryptographique sympathique. Considérez avoir une clé `key` (par exemple, un mot de passe) et un message secret `m`. Lorsque vous appliquez XOR sur ce message `m` avec la clé `key`, vous obtiendrez un message chiffré `cm` (c'est-à-dire, $cm = m \oplus key$), différent du premier message `m`. Pour revenir au message de base, il vous suffit de réappliquer la même opération sur votre message chiffré (c'est-à-dire, $m = cm \oplus key$). En résumé, si vous appliquez XOR deux fois avec la même clé, vous obtenez le message de départ, c'est-à-dire :

$$m = (m \oplus key) \oplus key$$

Trouvant la propriété amusante, vous décidez de créer un simple script Python vous permettant d'expérimenter un peu XOR. Dans un moment de folie, vous créez un fichier *binaire* aléatoire *très grand*, *tellement grand qu'il ne tient pas intégralement en mémoire*. Vous décidez que vous allez appliquer l'opérateur XOR sur ce fichier deux fois : une fois pour le chiffrer, une fois pour le déchiffrer (qui est exactement la même opération) afin de voir si vous obtenez réellement le fichier de départ.

Dans ce but, vous décidez avec vous-même la structure suivante :

1. Premièrement, vous créez une classe `XORApplier`, qui prendra une clé `key` comme paramètre du constructeur.
2. Ensuite, vous aurez une méthode vous permettant de lire votre fichier. À priori, cette méthode n'a pas d'utilité à être utilisée en dehors de votre classe.
3. Une seconde méthode vous permettra d'appliquer XOR sur l'argument donné. À nouveau, cette méthode n'a pas d'utilité à être utilisée en dehors de votre classe.
4. Enfin, la dernière méthode `apply(file, output)` lira le fichier `file`, le chiffrera avec XOR et écrira la sortie dans le fichier `output`.

L'opérateur XOR est implémenté par Python via le symbole `^`. Vous trouverez un exemple en bas de page.

Question 4.a. Commencez par implémenter la classe `XORApplier` comme décrit ci-dessus. Veillez à respecter les contraintes mémoire ainsi que les conventions de Python liées à l'encapsulation telles que vues en théorie et TP.

Dans le but de légèrement complexifier le chiffrement (mais surtout de vous amuser), vous décidez de refaire la même chose, mais à des endroits déterminés du fichier. Plus exactement, vous décidez d'utiliser une deuxième clé de chiffrement XOR aux positions $(c + d)$, où c est la position courante (0 au début) et d est un paramètre entier donné par l'utilisateur. Vous continuez à faire ceci tant que vous ne dépassez pas le fichier. Par exemple, avec $d = 3$ sur un fichier de taille 11, vous mettrez le chiffrement sur 0 (le départ), $0 + 3 = 3$, $3 + 3 = 6$ et $6 + 3 = 9$. Vous vous arrêtez là car $9 + 3 = 12$ dépasse le fichier.

Guillaume : La première partie me semble assez simple avec l'exemple. Mais dites-moi si la seconde partie rend l'examen trop long/complexe. PS : L'énoncé tient sur une page sans cette boîte en théorie. Hadrien : perso je pense que l'on peut se passer de cette partie. Globalement, l'examen me semble déjà suffisamment long. Par contre, je rajouterai dans la première partie une remarque pour dire que $\oplus$ peut être appliqué à une séquence de 0 et de 1 (ce n'est pas explicitement défini ci-dessus où $\oplus$ est explicité comme opérateur binaire entre deux valeurs 0 et/ou 1.

Question 4.b. Implémentez une méthode (en dehors de la classe `XORApplier`) qui effectue l'algorithme décrit ci-dessus. **Il vous est interdit de lire le fichier complet à nouveau. Vous devez vous déplacer dans le fichier à la bonne position à chaque fois.**

Pour rappel, vous trouverez ci-dessous un exemple simple de lecture/écriture binaire en Python ainsi que de l'utilisation de XOR.

```
1 # key vous est donne
2 # ouvre le fichier en mode lecture ET ecriture (r+)
3 with open("input.txt", 'r+b') as f: # 'b' pour mode Binaire
4     # lis le fichier complet
5     b = f.read()
6     # change les 3 premiers octets
7     out = [b[0] ^ key, b[1] ^ key, b[2] ^ key)
8     # ecrit le resultat du XOR dans le meme fichier
9     f.write(bytes(out)) # attention a bytes(...)
```

Réponse *(Utilisez les lignes pour spécifier les indentations)*